

GUIA DO PROJETO

EleiXan

Como executar o workspace Rust e o que cada parte do projeto faz

Estado analisado: 17 de setembro de 2026

Escopo: código presente neste repositório, com distinção entre arquitetura planejada e implementação atual.

1. Resumo rápido

O EleiXan foi organizado como um workspace Rust para uma urna universitária offline. A proposta separa a identificação do eleitor, feita pela mesa, do registro anônimo do voto, feito pela urna.

Ponto importante: a documentação descreve o sistema completo, mas o código atual ainda é uma prova de estrutura. Os módulos de criptografia, NFC, ledger e apuração ainda estão vazios.

O que existe hoje

- Um workspace Cargo com cinco crates independentes.
- Dois executáveis: `urna` e `apurar`.
- Três bibliotecas preparadas para receber a lógica: `core`, `nfc` e `mesa`.
- Documentação inicial de arquitetura, fluxo, casos de uso e hardware.

O que ainda não existe no código

- Leitura ou gravação real de NFC, PN532 ou NTAG215.
- Geração de UUID, HMAC-SHA256, hash chain, Ed25519 ou persistência em sled.
- Interface de votação, simulação de eleitores, lote de votos e verificação de ledger.
- Integração com Raspberry Pi ou Solana Devnet.

2. Como preparar o ambiente

2.1 Instalar Rust

No Windows, instale o Rust pelo instalador oficial **rustup**. Durante a instalação, aceite as opções padrão. Depois feche e abra o terminal novamente.

```
rustup --version
rustc --version
cargo --version
```

O primeiro comando confirma o gerenciador; os dois seguintes confirmam o compilador e o Cargo. Se aparecer “comando não reconhecido”, o Rust ainda não está no `PATH` daquele terminal.

2.2 Entrar na pasta do projeto

```
cd "C:\Users\Pedro\Desktop\Desenvolvimento\Aulas_Faculdade\EleiXan\EleiXan"
Get-ChildItem
```

Você deve ver `Cargo.toml`, `README.md` e a pasta `crates`.

2.3 Compilar o workspace

```
cargo check
cargo build
```

`cargo check` verifica rapidamente o código sem produzir um executável final. `cargo build` compila os artefatos. Para uma compilação otimizada, acrescente `--release`.

3. Executar o que já está implementado

3.1 Rodar a urna

```
cargo run -p eleixan-urna
```

Saída esperada:

```
EleiXan urna: modo simulacao ainda nao implementado
```

O pacote se chama `eleixan-urna`, mas o binário se chama `urna`. A opção `-p` escolhe o pacote do workspace.

3.2 Rodar a apuração

```
cargo run -p eleixan-apuracao
```

Saída esperada:

```
EleiXan apuracao: verificador ainda nao implementado
```

3.3 Compilar e executar em modo release

```
cargo build --release  
cargo run --release -p eleixan-urna  
cargo run --release -p eleixan-apuracao
```

Atenção ao README: os comandos `cargo run --release -- --mock-nfc --voters 870` e `cargo run --release --bin apurar ...` descrevem uma implementação futura. Hoje não há parser dessas opções nem um binário raiz com esse nome. Use os comandos desta seção.

4. O que cada arquivo faz

Local	Função atual	Função planejada
<code>Cargo.toml</code>	Declara o workspace e lista os cinco membros.	Centralizar versão, edição Rust 2021 e licença.
<code>crates/core</code>	Biblioteca vazia, com comentário de intenção.	Regras criptográficas, commitments, blocos e persistência do ledger.
<code>crates/nfc</code>	Biblioteca vazia.	Driver PN532, NTAG215, wipe e LEDs.
<code>crates/mesa</code>	Biblioteca vazia.	Emissão off-chain de tokens anônimos com HMAC.
<code>crates/urna/src/main.rs</code>	Imprime que a simulação ainda não foi implementada.	Executar o fluxo da cabine e registrar votos.
<code>crates/apuracao/src/main.rs</code>	Imprime que o verificador ainda não foi implementado.	Reconstruir a cadeia e conferir assinaturas.
<code>docs/</code>	Área reservada para documentação técnica.	SDD, ameaças, auditoria e decisões de arquitetura.
<code>hardware/</code>	README reservado para hardware.	Esquemas, lista de materiais e montagem.
<code>scripts/</code>	README reservado para scripts.	Simulação de ataques, carga e QA.

5. Arquitetura planejada, passo a passo

1. Mesa

Valida a matrícula no registro local e emite um token descartável. A identidade não deve ser gravada no token.

2. Token

O desenho prevê UUID v4 e assinatura HMAC-SHA256 armazenados em uma tag NTAG215.

3. Urna

O leitor PN532 valida o token e verifica se ele já foi usado. Um HashSet representa o controle de uso único.

4. Voto

O voto deve virar um commitment com SHA-256 e salt, sem transportar a identidade do eleitor.

5. Ledger

Os blocos planejados encadeiam o hash anterior e recebem assinatura Ed25519 da urna.

6. Reset

A tag deve ser zerada e relida. LED verde indica sucesso; LED vermelho indica falha.

7. Apuração

O cartão SD é levado ao apurador, que reconstrói a cadeia e verifica assinaturas.

8. Prova pública

Opcionalmente, o hash final pode ser ancorado na Solana Devnet.

Por que o batch é mencionado?

A proposta mantém commitments temporariamente em RAM e grava em lotes, por exemplo a cada dez votos ou após um intervalo aleatório. Isso tenta dificultar a correlação entre a ordem de chegada à cabine e a ordem dos blocos. Esse mecanismo é descrito no README, mas ainda não foi implementado.

6. Como interpretar o estado do projeto

Comandos úteis

```
cargo test
cargo tree
cargo metadata --no-deps
```

Hoje, `cargo test` provavelmente termina sem testes executáveis porque não existem testes declarados. `cargo tree` mostra que os crates ainda não têm dependências externas. `cargo metadata --no-deps` exhibe a estrutura formal do workspace.

Limitações atuais

- O projeto compila apenas depois que o toolchain Rust estiver instalado.
- A documentação é mais avançada que a implementação.
- Não é possível realizar uma eleição real ou uma simulação completa nesta versão.
- Não conecte hardware nem use dados reais esperando que o sistema os processe.

Resumo: para apresentar o estado atual, rode `cargo check`, depois os dois comandos `cargo run -p ...`. Eles comprovam que a estrutura do workspace está criada, enquanto as mensagens de saída deixam claro o que ainda falta.

7. Próximos passos recomendados

1. Adicionar testes unitários para hashing, tokens e validação de uso único.
2. Implementar o modelo de dados do ledger em `core` antes de conectar hardware.
3. Criar um driver NFC simulado em `nfc` para permitir testes sem Raspberry Pi.
4. Implementar a mesa e a urna usando traits, mantendo o modo mock separado do PN532 real.
5. Implementar o binário de apuração e testes de adulteração de blocos.
6. Atualizar o README quando os comandos descritos passarem a existir.

Conclusão

O repositório já estabelece a divisão de responsabilidades e a estrutura modular de uma urna offline. O próximo trabalho é transformar cada comentário de intenção em uma API testável, começando pelo núcleo criptográfico e pelo modo simulado.

Documento gerado a partir da leitura do código e da documentação presentes no workspace EleiXan.